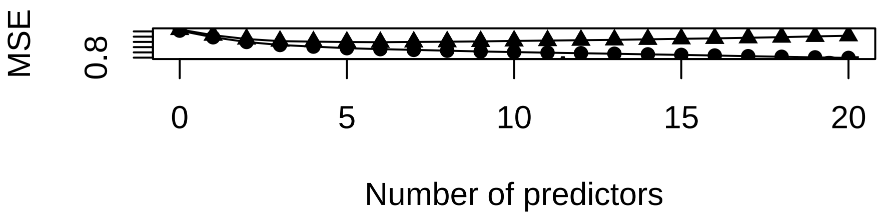
# Question 1(a)

```r
set.seed(43202)

n <- 100
p <- 20
nsim <- 200
sigma <- 1

X_all <- matrix(rnorm(n * p), nrow = n, ncol = p)
covariate_count <- 0:p

beta <- 0.4^sqrt(1:p)
mu <- as.vector(X_all %*% beta)

train_mse <- matrix(NA_real_, nrow = nsim, ncol = p + 1)
test_mse <- matrix(NA_real_, nrow = nsim, ncol = p + 1)

for (k in 1:nsim) {

  y_train <- mu + rnorm(n, sd = sigma)
  y_test <- mu + rnorm(n, sd = sigma)

  y_hat <- rep(mean(y_train), n)
  train_mse[k, 1] <- mean((y_train - y_hat)^2)
  test_mse[k, 1] <- mean((y_test - y_hat)^2)

  for (m in 1:p) {

    train_data <- data.frame(
      y = y_train,
      X_all[, 1:m, drop = FALSE]
    )

    test_data <- data.frame(
      X_all[, 1:m, drop = FALSE]
    )

    fit <- lm(y ~ ., data = train_data)
    y_hat <- predict(fit, newdata = test_data)

    train_mse[k, m + 1] <- mean(residuals(fit)^2)
    test_mse[k, m + 1] <- mean((y_test - y_hat)^2)
  }
}

mean_train_mse <- colMeans(train_mse)
mean_test_mse <- colMeans(test_mse)

all(apply(
  train_mse,
  1,
  function(x) all(diff(x) <= 1e-10)
))
```

```
## [1] TRUE
```

```r
plot(
  covariate_count,
  mean_train_mse,
  type = "o",
  pch = 16,
  xlab = "Number of predictors",
  ylab = "MSE",
  ylim = range(mean_train_mse, mean_test_mse)
)

lines(
  covariate_count,
  mean_test_mse,
  type = "o",
  pch = 17
)

legend(
  "topright",
  legend = c(
    "Average training MSE",
    "Average test MSE"
  ),
  lty = 1,
  pch = c(16, 17),
  bty = "n"
)
```




As predictors are added, the average training MSE would decrease constantly while the average test MSE possibly fluctate.

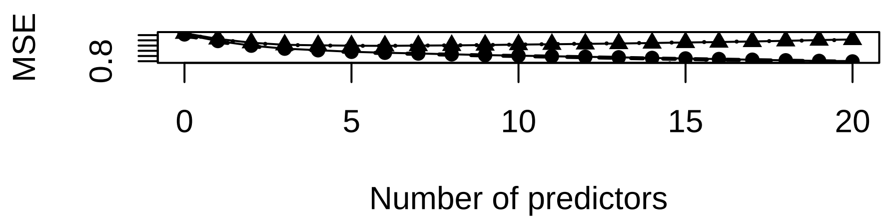
The average training MSE decrease continuously since larger least squares model contains the smaller one (the new prdictors' coefficient can be set as 0)

The test MSE may increase because new predictors may fit random noise rather than true signal. Traning error decreases indeed while the variance increases, which not necessarily improve prediction.

## Question 1(b)

```r
theory_train_mse <- numeric(p + 1)
theory_test_mse <- numeric(p + 1)

for (m in 0:p) {

  if (m == 0) {
    X_m <- matrix(1, nrow = n, ncol = 1)
  } else {
    X_m <- cbind(
      1,
      X_all[, 1:m, drop = FALSE]
    )
  }

  fit_mu <- lm.fit(
    x = X_m,
    y = mu
  )

  mu_hat <- fit_mu$fitted.values

  bias_mse <- mean(
    (mu - mu_hat)^2
  )

  theory_train_mse[m + 1] <-
    bias_mse + sigma^2 * (1 - (m + 1) / n)

  theory_test_mse[m + 1] <-
    bias_mse + sigma^2 * (1 + (m + 1) / n)
}

plot(
  covariate_count,
  mean_train_mse,
  type = "o",
  pch = 16,
  xlab = "Number of predictors",
  ylab = "MSE",
  ylim = range(
    mean_train_mse,
    mean_test_mse,
    theory_train_mse,
    theory_test_mse
  )
)

lines(
  covariate_count,
  mean_test_mse,
  type = "o",
  pch = 17
)

lines(
  covariate_count,
  theory_train_mse,
  lty = 2,
  lwd = 2
)

lines(
  covariate_count,
```

```r
  theory_test_mse,
  lty = 3,
  lwd = 2
)

legend(
  "topright",
  legend = c(
    "Average training MSE",
    "Average test MSE",
    "Expected training MSE",
    "Expected test MSE"
  ),
  lty = c(1, 1, 2, 3),
  pch = c(16, 17, NA, NA),
  bty = "n"
)
```

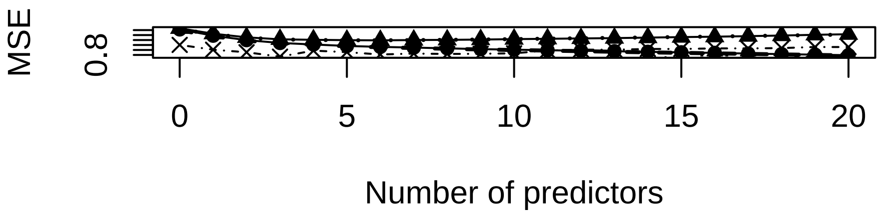



The simulation averages are close to their theoretical expectations. The expected test MSE reflects a tradeoff between approximation bias, which generally decreases as predictors are added, and estimation variance, which increases with model size.

## Question 1(c)

```r
single_test_mse <- test_mse[1, ]

plot(
  covariate_count,
  mean_train_mse,
  type = "o",
  pch = 16,
  xlab = "Number of predictors",
  ylab = "MSE",
  ylim = range(
    mean_train_mse,
    mean_test_mse,
    theory_train_mse,
    theory_test_mse,
    single_test_mse
  )
)

lines(
  covariate_count,
  mean_test_mse,
  type = "o",
  pch = 17
)

lines(
  covariate_count,
  theory_train_mse,
  lty = 2,
  lwd = 2
)

lines(
  covariate_count,
  theory_test_mse,
  lty = 3,
  lwd = 2
)

lines(
  covariate_count,
  single_test_mse,
  type = "o",
  pch = 4,
  lty = 4
)

legend(
  "topright",
```

```r
  legend = c(
    "Average training MSE",
    "Average test MSE",
    "Expected training MSE",
    "Expected test MSE",
    "Single-run test MSE"
  ),
  lty = c(1, 1, 2, 3, 4),
  pch = c(16, 17, NA, NA, 4),
  bty = "n"
)
```




The single-run test MSE is less smooth because it contains the random variation from one particular test response.

Training MSE cannot be used by itself to choose the predictor count because it tends to decrease as the model becomes more flexible. Expected test MSE balances decreasing approximation bias with increasing estimation variance, so adding more predictors does not necessarily improve prediction.

## Question 2(a)

```r
set.seed(43203)

n <- 100
q <- 6
ncount <- 200
sigma <- 1

i <- 1:n

X_all <- sapply(1:q, function(k) {
  sqrt(2) * cos(pi * k * (i - 0.5) / n)
})

beta <- rep(0.5, q)

x0 <- c(0, 1, 0, 0, 1, 0)

mu0 <- sum(x0 * beta)
mu0
```

```
## [1] 1
```

```r
squared_bias <- numeric(q + 1)
estimation_variance <- numeric(q + 1)
theoretical <- numeric(q + 1)

for (m in 0:q) {

  if (m < q) {
    bias <- sum(
      x0[(m + 1):q] * beta[(m + 1):q]
    )
  } else {
    bias <- 0
  }

  squared_bias[m + 1] <- bias^2

  if (m == 0) {
    estimation_variance[m + 1] <- sigma^2 / n
  } else {
    estimation_variance[m + 1] <-
      sigma^2 / n *
      (1 + sum(x0[1:m]^2))
  }

  theoretical[m + 1] <-
    squared_bias[m + 1] +
```

```r
    estimation_variance[m + 1]
}

theory_results <- data.frame(
  m = 0:q,
  squared_bias = squared_bias,
  estimation_variance = estimation_variance,
  expected_error = theoretical
)

theory_results
```

```
##   m squared_bias estimation_variance expected_error
## 1 0         1.00                0.01           1.01
## 2 1         1.00                0.01           1.01
## 3 2         0.25                0.02           0.27
## 4 3         0.25                0.02           0.27
## 5 4         0.25                0.02           0.27
## 6 5         0.00                0.03           0.03
## 7 6         0.00                0.03           0.03
```

The target mean is

$$\mu_0 = x_0^\top \beta = 1.$$

The theoretical expected squared errors for $m = 0, \ldots, 6$ are

$$1.01,\ 1.01,\ 0.27,\ 0.27,\ 0.27,\ 0.03,\ 0.03.$$

The squared bias comes from the contributions of omitted predictors at the target point. The variance term comes from estimating the intercept and the included coefficients. Because the predictor columns are orthogonal, these variance contributions add.

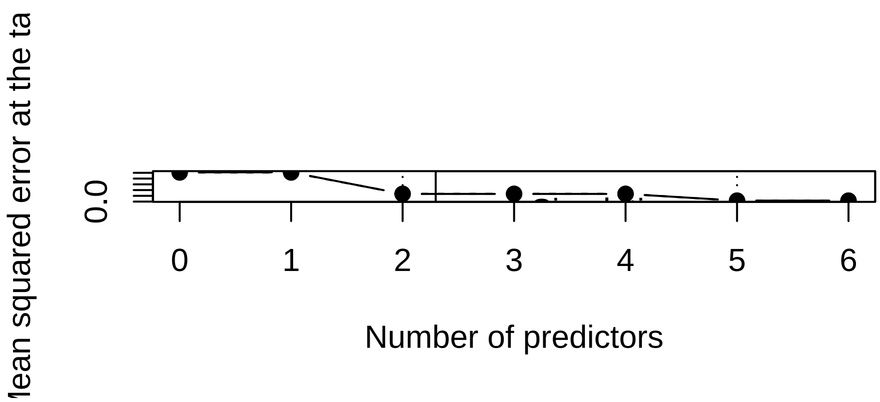
The expected error changes when predictors 2 and 5 are added and remains unchanged when predictors 1, 3, 4, and 6 are added.

# Question 2(b)

```r
sq_error <- matrix(
  NA,
  nrow = ncount,
  ncol = q + 1
)

for (b in 1:ncount) {

  y <- as.vector(
    X_all %*% beta + rnorm(n, sd = sigma)
  )

  for (m in 0:q) {

    if (m == 0) {

      X_m <- matrix(1, nrow = n, ncol = 1)
      x0_m <- 1

    } else {

      X_m <- cbind(
        1,
        X_all[, 1:m, drop = FALSE]
      )

      x0_m <- c(
        1,
        x0[1:m]
      )
    }

    beta_hat <- solve(
      crossprod(X_m),
      crossprod(X_m, y)
    )

    mu_hat <- sum(x0_m * beta_hat)

    sq_error[b, m + 1] <-
      (mu_hat - mu0)^2
  }
}

simulated <- colMeans(sq_error)

results <- data.frame(
  m = 0:q,
  simulated = simulated,
  theoretical = theoretical
)

results
```

```
##   m  simulated theoretical
## 1 0 1.01891864        1.01
## 2 1 1.01891864        1.01
## 3 2 0.26711699        0.27
## 4 3 0.26711699        0.27
## 5 4 0.26711699        0.27
## 6 5 0.03375873        0.03
## 7 6 0.03375873        0.03
```

```r
plot(
  0:q,
  simulated,
  type = "b",
  pch = 19,
  xlab = "Number of predictors",
  ylab = "Mean squared error at the target",
  ylim = range(c(simulated, theoretical))
)

lines(
  0:q,
  theoretical,
  type = "b",
  lty = 2
)

abline(
  v = c(2, 5),
  lty = 3
)

legend(
  "topright",
  legend = c(
    "Simulation average",
    "Theoretical expectation"
  ),
  lty = c(1, 2),
  pch = c(19, NA)
)
```




The simulated averages are close to the theoretical expected squared errors, as shown in the table and plot.

The smallest predictor count that minimizes the theoretical error is $m = 5$.

## Question 2(c)

The error changes only when predictors 2 and 5 are added because these are the only nonzero coordinates of the target point. The other target coordinates are zero, so their fitted coefficients are multiplied by zero in this prediction.

This is a target-specific conclusion. A predictor can still matter for average test MSE if its values are nonzero at other points in the design.

## Question 3(a)

The training residual is

$$\begin{aligned}\mathbf{y} - \mathbf{H}\mathbf{y} &= (\mathbf{I}_n - \mathbf{H})(\boldsymbol{\mu} + \boldsymbol{\epsilon}) \\ &= (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + (\mathbf{I}_n - \mathbf{H})\boldsymbol{\epsilon}.\end{aligned}$$

The cross term has expectation zero. Since $\mathbf{I}_n - \mathbf{H}$ is symmetric and idempotent,

$$\begin{aligned}E\left(\|\mathbf{y} - \mathbf{H}\mathbf{y}\|_2^2 \mid \mathbf{X}\right) &= B^2 + \sigma^2 \operatorname{tr}(\mathbf{I}_n - \mathbf{H}) \\ &= B^2 + \sigma^2(n - (p+1)).\end{aligned}$$

Dividing by $n$,

$$E(\mathrm{MSE}_{\mathrm{train}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 - \frac{p+1}{n}\right).$$

For the independent test response,

$$\mathbf{y}^* - \mathbf{H}\mathbf{y} = (\mathbf{I}_n - \mathbf{H})\boldsymbol{\mu} + \boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}.$$

Again, all cross terms have expectation zero. By independence,

$$\begin{aligned} E\left(\|\boldsymbol{\epsilon}^* - \mathbf{H}\boldsymbol{\epsilon}\|_2^2 \mid \mathbf{X}\right) &= n\sigma^2 + \sigma^2 \operatorname{tr}(\mathbf{H}) \\ &= \sigma^2(n + (p+1)). \end{aligned}$$

Therefore,

$$E(\mathrm{MSE}_{\mathrm{test}} \mid \mathbf{X}) = \frac{B^2}{n} + \sigma^2\left(1 + \frac{p+1}{n}\right).$$

# Question 3(b)

Subtracting the two expectations gives the expected optimism:

$$E\left(\mathrm{MSE}_{\mathrm{test}} - \mathrm{MSE}_{\mathrm{train}} \mid \mathbf{X}\right) = \frac{2(p+1)\sigma^2}{n}.$$

For

$$n = 120, \qquad p = 4, \qquad \frac{B^2}{n} = 0.04, \qquad \sigma^2 = 1,$$

the expected training MSE is

$$\begin{aligned} E(\mathrm{MSE}_{\mathrm{train}} \mid \mathbf{X}) &= 0.04 + 1\left(1 - \frac{4+1}{120}\right) \\ &= 0.998333. \end{aligned}$$

The expected test MSE is

$$\begin{aligned} E(\mathrm{MSE}_{\mathrm{test}} \mid \mathbf{X}) &= 0.04 + 1\left(1 + \frac{4+1}{120}\right) \\ &= 1.081667. \end{aligned}$$

Their expected difference is

$$1.081667 - 0.998333 = 0.083333.$$

These quantities are long-run expectations. For one particular pair of training and test responses, random variation can make the realized $\mathrm{MSE}_{\mathrm{test}}$ smaller than the realized $\mathrm{MSE}_{\mathrm{train}}$.

# Question 3(c)

The corrected estimate of test MSE is

$$\begin{aligned} \widehat{\mathrm{MSE}}_{\mathrm{test}} &= \frac{\mathrm{RSS} + 2(p+1)\widehat{\sigma}^2}{n} \\ &= \frac{116.4 + 2(4+1)(0.96)}{120} \\ &= 1.05. \end{aligned}$$

Mallows' $C_p$ is

$$\begin{aligned} C_p &= \frac{\mathrm{RSS}}{\widehat{\sigma}^2} - n + 2(p+1) \\ &= \frac{116.4}{0.96} - 120 + 2(4+1) \\ &= 11.25. \end{aligned}$$

The relationship between the two quantities is

$$
\begin{aligned}
\frac{\widehat{\sigma}^2(C_p+n)}{n} &= \frac{0.96(11.25+120)}{120} \\
&= 1.05 \\
&= \frac{\text{RSS} + 2(p+1)\widehat{\sigma}^2}{n}.
\end{aligned}
$$

Therefore,

$$
\widehat{\text{MSE}}_{\text{test}} = \frac{\widehat{\sigma}^2(C_p+n)}{n}.
$$

When $n$ and $\widehat{\sigma}^2$ are common to all candidate models, this is an increasing transformation of $C_p$. Therefore, minimizing $C_p$ is equivalent to minimizing the corrected test MSE.

# Question 4(a)

```r
data_candidates <- c(
  "data/diabetes.csv",
  "../../../data/week-02/diabetes.csv",
  "data/week-02/diabetes.csv"
)

data_file <- data_candidates[file.exists(data_candidates)][1]
stopifnot(!is.na(data_file))

training <- read.csv(
  data_file,
  check.names = FALSE
)[1:370, ]

n <- nrow(training)

model_variables <- list(
  A = c(
    "bmi", "bp", "s5",
    "sex", "s1", "s2", "s4"
  ),

  B = c(
    "bmi", "bp", "s5",
    "sex", "s1", "s2", "s4", "s6"
  ),

  Full = c(
    "age", "sex", "bmi", "bp", "s1",
    "s2", "s3", "s4", "s5", "s6"
  )
)

fit_rss <- function(variables) {

  X <- cbind(
    1,
    as.matrix(training[variables])
  )

  fit <- lm.fit(
    X,
    training$y
  )

  sum(fit$residuals^2)
}

rss <- vapply(
  model_variables,
  fit_rss,
  numeric(1)
)

p <- vapply(
  model_variables,
  function(variables) length(variables),
  numeric(1)
)

residual_df <- n - (p["Full"] + 1)

sigma2_hat <- rss["Full"] / residual_df

results_a <- data.frame(
  model = c(
    "Model A",
    "Model B",
    "Full"
  ),
  p = unname(
```

```r
    p[c("A", "B", "Full")]
  ),
  RSS = unname(
    rss[c("A", "B", "Full")]
  )
)

results_a
```

```
##     model  p     RSS
## 1 Model A  7 1098909
## 2 Model B  8 1089717
## 3    Full 10 1089452
```

```r
residual_df
```

```
## Full
##  359
```

```r
sigma2_hat
```

```
##     Full
## 3034.684
```

The full model gives

$$\widehat{\sigma}^2 = 3034.684460.$$

# Question 4(b)

```r
results_b <- data.frame(
  model = c(
    "Model A",
    "Model B"
  ),

  p = unname(
    p[c("A", "B")]
  ),

  RSS = unname(
    rss[c("A", "B")]
  )
)

results_b$Cp <-
  results_b$RSS / sigma2_hat -
  n +
  2 * (results_b$p + 1)

results_b$AIC <-
  n * log(
    results_b$RSS / n
  ) +
  2 * (results_b$p + 1)

results_b$BIC <-
  n * log(
    results_b$RSS / n
  ) +
  log(n) * (results_b$p + 1)

results_b
```

```
##     model p     RSS       Cp      AIC      BIC
## 1 Model A 7 1098909 8.116253 2974.640 3005.948
## 2 Model B 8 1089717 7.087434 2973.532 3008.754
```

$C_p$ selects Model B, reduced AIC selects Model B, and reduced BIC selects Model A.

# Question 4(c)

```r
fit_gain <- n * log(
  rss["A"] / rss["B"]
)

cp_gain <- (
```

```r
  rss["A"] - rss["B"]
) / sigma2_hat

aic_penalty <- 2
bic_penalty <- log(n)
cp_penalty <- 2

comparison <- data.frame(
  criterion = c(
    "AIC",
    "BIC",
    "Cp"
  ),

  fit_improvement = c(
    fit_gain,
    fit_gain,
    cp_gain
  ),

  penalty = c(
    aic_penalty,
    bic_penalty,
    cp_penalty
  )
)

comparison
```

```
##   criterion fit_improvement  penalty
## 1       AIC        3.107775 2.000000
## 2       BIC        3.107775 5.913503
## 3        Cp        3.028819 2.000000
```

Adding predictor `s6` improves the fit, but the criteria use different penalties for model complexity. The improvement is large enough for $C_p$ and AIC to prefer Model B, while BIC uses a larger penalty and therefore prefers Model A.

AIC, BIC, and $C_p$ only select the model that is more appropriate under a particular criterion. They cannot prove which model is the true data-generating model since it only have the current sample, some important predictors may be missing from the model, and the true relationship may be nonlinear.

# Question 5(a)

Step 2 involves an improper use of the final test data because the test results were used to compare the 11 candidate models and determine the number of predictors.

Due to the limited sample size, each test mean squared error (MSE) contains random fluctuation. Selecting the minimum value from the 11 test MSEs might simply pick the model that happened to have the lowest error by chance, leading to an overly optimistic estimate of future prediction error.

# Question 5(b)

An effective approach to splitting the current 442 rows of data into six groups is as follows: reserve rows 371 through 442 as the final test set, and use rows 1 through 370 for five-fold cross-validation.

For each number of predictors, fit the model using four of the folds and calculate the MSE on the remaining fold. Repeat this process five times. Average the five validation MSE values and select the number of predictors that yields the lowest average validation MSE. Once the number of predictors is determined, refit the model using all 370 training observations. Finally, evaluate the fitted model using the data from rows 371–442.

# Question 5(c)

If all 11 final test MSE values have already been examined, the test set has effectively taken on the role of a validation set. While the results can still be reported as part of an exploratory analysis, the minimum MSE can no longer be considered an unbiased, independent estimate of future prediction error (a new test dataset would be required for that).